

BciPy 2.0: Experiment Orchestration, Multimodal Support, and Simulations in Python.

Tab Memmott^{1,2*}, Matthew Lawhead^{3*}, Basak Celik^{4*}, Niklas Smedemark-Margulies^{4*}, Dylan Gaines^{5,6*}, Daniel Klee^{1*}, Carson Reader^{2*}, Allen Zhang^{2*}, Srikar Ananthoju^{4*}, and Keith Vertanen^{5*}

¹ Department of Neurology, Oregon Health & Science University, Portland, OR, United States of America ² Institute on Development and Disability, Oregon Health & Science University, Portland, OR, United States of America ³ Oregon Clinical and Translational Research Institute, Oregon Health & Science University, Portland, OR, United States of America ⁴ Department of Electrical and Computer Engineering, Northeastern University, Boston, MA, United States of America ⁵ Department of Computer Science, Michigan Technological University, Houghton, MI, United States of America ⁶ Department of Computer Science, Kennesaw State University, Marietta, GA, United States of America ¶ Corresponding author * These authors contributed equally.

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: 

Submitted: 05 May 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](https://creativecommons.org/licenses/by/4.0/)).

Summary

Advances in Brain-Computer Interface (BCI) research require software that evolves alongside new scientific discoveries and experimental needs. BciPy 2.0 is a major update and expansion of the original BciPy 1.0, developed in response to recent progress in the field. This release addresses growing demands for multimodal integration, offline simulation, and reproducible experimental protocols, with a particular focus on communication BCIs (cBCIs). BciPy 2.0 offers robust support for multimodal data acquisition and fusion, advanced simulation capabilities, flexible task orchestration, and standardized data sharing—all within the Python ecosystem. The system prioritizes modularity and extensibility, incorporating features informed by current research trends. This manuscript provides a comprehensive overview of BciPy 2.0, including system architecture and practical usage examples.

Statement of Need

Software is the foundation of BCI research, serving as the bridge between biosignals and real-time applications that enable computer-mediated control. Reliable and adaptable tools are crucial for improving system accuracy, reducing latency, and expanding functionality—especially for communication BCI (cBCI) applications. These improvements bring cBCIs closer to practical, real-world use, with significant implications for healthcare, accessibility, and human-computer interaction.

Current trends in BCI research include increased interest in multimodal signal acquisition and integration, as well as the use of advanced modeling techniques to improve classification and inference. The scientific community is also prioritizing data practices that follow the FAIR principles — Findable, Accessible, Interoperable, and Reusable (Wilkinson et al., 2016) — which BciPy 2.0 is built to support. Additionally, some dependencies and Python versions used in BciPy 1.0 are now deprecated or incompatible with modern tools, motivating the architectural changes in BciPy 2.0. Future releases will continue to enhance interoperability with popular scientific libraries and features, and provide expanded support for experiment management.

41 State of the Field

42 A large portion of the noninvasive BCI field relies on custom-built software or established
43 frameworks such as BCI2000 (Schalk et al., 2004) and OpenViBE (Renard et al., 2010),
44 which provide broad support for applications like cursor control and virtual reality (VR)
45 integration but are written in C++ and oriented toward general BCI paradigms rather than
46 communication-specific workflows. More recent Python packages such as PyBCI (Booth et
47 al., 2023), MetaBCI (Mei et al., 2024), and PyNoetic (Singh et al., 2025) offer EEG signal
48 acquisition, artifact handling, and interface control, but lack integrated support for multimodal
49 evidence fusion, language modeling, and offline simulation. BciPy occupies a distinct niche
50 by focusing specifically on text input for communication BCIs, combining these capabilities
51 within a single Python framework. Python's dominance in scientific computing and machine
52 learning allows BciPy to leverage a broad ecosystem of libraries and tools, making it accessible
53 to researchers who may not have extensive programming experience.

54 Software Design

55 BciPy supports installation on the latest versions of macOS, Linux, and Windows, with step-
56 by-step instructions provided in the documentation and reproducible builds verified through
57 continuous integration with GitHub Actions. Each submodule includes its own README.md,
58 runnable demos, and unit tests to help users get started. Users can interact with BciPy
59 through the client interface, by importing the package in Python, or via the PyQt6-based
60 GUI (BCInterface.py, see Figure 1). The choice of interface depends on the user's coding
61 experience and the level of customization required for their experiments.

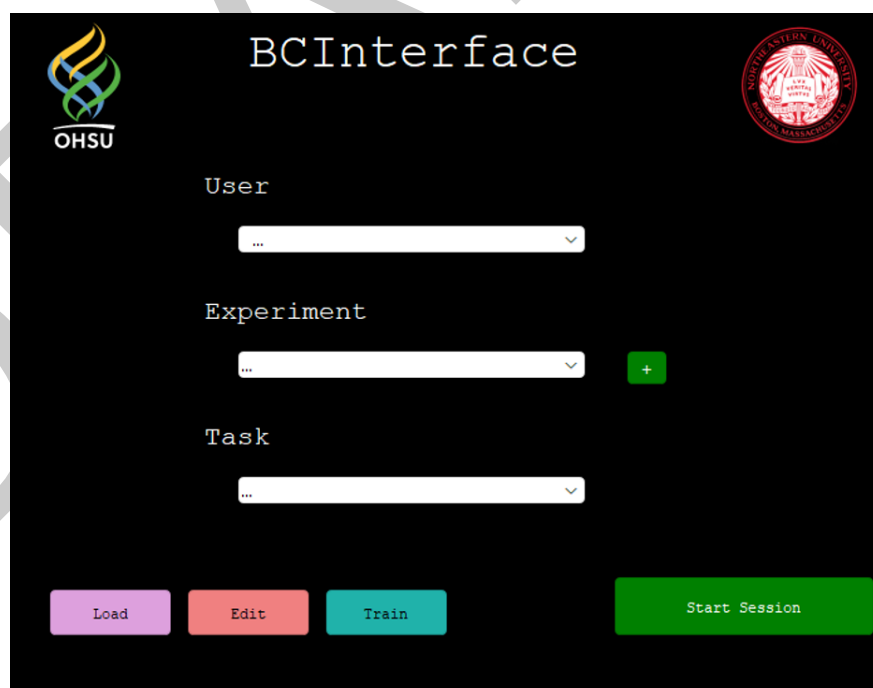


Figure 1: BciPy GUI. The BciPy GUI can be used for editing or loading parameters, training a SignalModel, defining a new experiment (this provides another GUI), or running an experiment or Task.

62 Experiment parameters are defined in JSON format, with default templates available in
63 bciPy/parameters/. These parameters can be edited directly in the JSON files or through
64 the graphical parameter editor shown in Figure 2, allowing researchers to configure experiment

65 conditions upfront and reduce input errors. Data collected with BciPy can be exported to
66 multiple formats—including BDF, EDF, BrainVision, Brain Imaging Data Structure (BIDS),
67 and MNE—for external analysis or sharing, with compression options to facilitate storage.
68 These export capabilities support FAIR data principles and interoperability with common
69 analysis tools.

70 BciPy leverages several scientific libraries to provide its core features, including PsychoPy,
71 PyLSL, scikit-learn, transformers, NumPy, SciPy, Pandas, and MNE (Gramfort et al., 2014;
72 Kothe et al., 2025; McKinney & others, 2011; Peirce, 2007; Van Der Walt et al., 2011;
73 Virtanen et al., 2020; Wolf et al., 2020). The full list of dependencies is maintained in the
74 pyproject.toml file.

Editing: C:/Users/CAMBI-research/BciPy/bcipy/parameters/parameters.json

Search: Clear

Changed Parameters: [+]

Fake EEG Data
If 'true', fake EEG data will be used instead of real EEG data. Useful for testing by software development team.
☐ Editable
☐ Enable Fake EEG Data

Acquisition Mode
Specifies the hardware device(s) used for data collection. Default: EEG.
☐ Editable
EEG

Trigger Stimulus Type
Specifies whether to use text, image or auditory stimulus to calibrate trigger latency. Default: text
☐ Editable
text

Number of Cross-Validation Folds
Specifies the number of folds used for cross-validation when calculating AUC. Default: 10
☐ Editable
10

Save Save As Open

Figure 2: BciPy Parameter Editor. The BciPy Parameter Editing GUI can be used for editing, saving, or searching a parameters file. This can help prevent input errors and facilitate defining parameters for experiment conditions upfront. If parameters are changed, a panel under Changed Parameters (shown above) will display with the parameter changed and what value it's been updated to. This can help prevent accidental changes or debug issues with a set of parameters.

Research Impact Statement

The BciPy repository has made BCI research more accessible through a modular, extensible, real-time Python interface designed for practical use and reproducible experimentation. The software and accompanying documentation have been released publicly, allowing researchers to directly run the system and adapt the interface for their own BCI studies by adding new paradigms and processing methods. The repository includes example pipelines, standardized data handling utilities, and integration with common Python scientific libraries, which has helped lower the technical barrier for working with neural signals in real time.

Evidence of use is reflected in 76,000 estimated downloads (from PyPI) and 39 external forks that adapt the interface for related BCI experiments and prototyping workflows. The BciPy Python library has also been used internally by 18 developers and by collaborators to build and test closed-loop BCI applications, demonstrating that the interface is stable enough for real experimental setups rather than only proof-of-concept demonstrations. Since its initial public release (Memmott et al., 2021), BciPy has gained wide adoption in the BCI community, with 145 GitHub stars, approximately 30 citations across peer-reviewed and preprint venues. There are also more than 10 peer-reviewed publications that have used BciPy in control and clinical studies (a full list is maintained in the repository README.md). Early adoption of BciPy 2.0 is evident, with five published studies utilizing its redesigned architecture as of March 2026. The Python BCI ecosystem has also grown, with several complementary libraries released or updated (Booth et al., 2023; Mei et al., 2024; Singh et al., 2025; Zhu et al., 2024).

BciPy is positioned for near-term impact within the BCI community due to its emphasis on reproducibility, clear documentation, and compatibility with common hardware, software, and analysis tools. By providing a simple and extensible interface for neural data acquisition and control, our work helps accelerate rapid prototyping and experimental iteration in BCI research.

BciPy 2.0 Overview

BciPy 2.0 is a major update to the original BciPy 1.0, with significant improvements in architecture, functionality, and usability. The following sections describe the key additions in detail: task and experiment orchestration, multimodal data acquisition and evidence fusion, and a new simulation module for offline evaluation of system parameters.

Task & Experiment Support

BciPy manages the execution of experimental Tasks using the SessionOrchestrator class (see Figure 3), which ensures Tasks are run in the correct order and that all data are properly persisted. Researchers can run individual Tasks, such as Calibration, directly via the client, or define complete experimental protocols for reproducible studies.

Experiment protocols are specified in the experiments.json file as ordered sequences of Tasks. The orchestrator reads this protocol, executes each Task sequentially, and logs all relevant information. Each Task writes its own logs to a dedicated subdirectory for easy tracking and analysis.

In addition to standard Tasks, protocols can include Actions, which are lightweight subclasses of Task. Actions do not require display or data acquisition and are useful for simple steps such as prompting the researcher with a dialog or indicating experiment progress. For example, an IntertaskAction can be inserted to request input from the researcher before continuing to the next Task in the sequence.

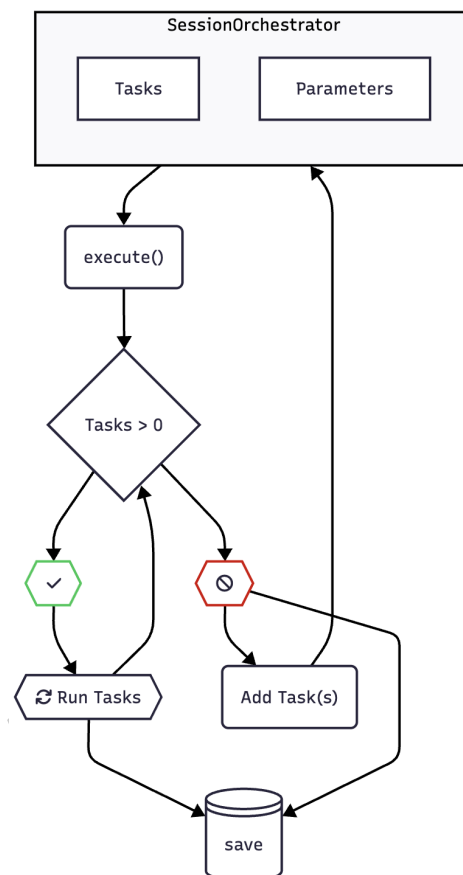


Figure 3: Session Orchestration. The SessionOrchestrator executes a sequence of Tasks defined in an experiment protocol. Each Task is initialized with the current parameters and any data needed from previous Tasks. The orchestrator manages the flow of data between Tasks and ensures that each Task is executed in the correct order. The SessionOrchestrator, once initialized with parameters and optional metadata, is ready for Tasks to be added using the `add_tasks()` or `add_task()` methods. These can be defined and loaded using the `experiment.json` and defined protocol or manually added to the SessionOrchestrator. The experiment can then be run using `execute()`. This method loops over Tasks, providing all parameters and a log needed for operation. The Tasks are then responsible for initializing any objects required for operation, such as the `DataAcquisitionClient`, `Display`, or `LanguageModel`. After each Task and the entire execution loop, the data persists on disk.

Multimodal Data Acquisition and Fusion

BciPy 2.0 introduces support for multimodal data acquisition and evidence fusion. The system can consider information from multiple devices when making typing decisions. The BciPy 1.0 data acquisition module supported TCP-based connections as well as connections through LabStreamingLayer (LSL) (Kothe et al., 2025). After extensive testing, we leaned more heavily on LSL to support multimodal acquisition in BciPy 2.0. LSL is well-supported across the industry, with many devices providing compatible drivers. This decision allowed us to drastically simplify the acquisition module while increasing functionality.

The data acquisition module in BciPy has two primary responsibilities: passively recording streaming device data to disk for later processing, and querying data in real time for use in a typing task. BciPy 2.0 streamlines the query interface from BciPy 1.0, replacing the previous unconstrained session-wide data access with a more efficient, event-driven approach suited to

130 multimodal workflows.

131 The approach for combining multimodal data sources is described briefly. Under the conditional
132 independence assumption, a Bayesian probabilistic algorithm that fuses information from
133 multiple sources of evidence is employed to achieve multimodal classification. The posterior
134 probabilities for user intent (θ) given biosignals data ($x_{1:s}$) are computed using Bayes' rule as
135 follows:

$$p(\theta | x_{1:s}) \propto p(\theta) \prod_{j=1}^s p(x_j | \theta), \quad (1)$$

136 where s denotes the number of sources ($s > 1$ for multimodal) and $p(\theta)$ is the class prior.

137 In addition to the EEG signal model developed in BciPy 1.0, BciPy 2.0 introduces a gaze
138 model for classification of eye gaze trajectory data that can be acquired in parallel with the
139 EEG data stream through an eye tracker. Both the EEG and gaze models inherit the same
140 SignalModel structure and are compatible with the scikit-learn estimators API (Pedregosa et
141 al., 2011).

142 The provided gaze model assumes positional and temporal dependence in gaze data. It is also
143 assumed that the gaze trajectory (x_g) obeys a Gaussian Process distribution, that is:

$$p(x_g | \theta) \sim \mathcal{GP}(\mu_{g,\theta}, \Sigma_g), \quad (2)$$

144 where $\mu_{g,\theta}$ and Σ_g are the multidimensional means and shared covariances corresponding to
145 the class labels, respectively. Details of the multimodal fusion method for EEG and gaze data
146 are further described in the repository.

147 BciPy 2.0 expands upon the language modeling capabilities of BciPy 1.0 and removes the
148 language model (LM) module from the previous Docker image, opting instead for direct
149 function calls in Python. The goal of the language model remains the same as the prior
150 version—to accelerate the text input task by providing additional evidence to the system. The
151 LM module takes the context, or the text that the user has written so far, and produces an
152 initial likelihood distribution over the system's symbol set. This distribution can be used to
153 present more likely characters to the user sooner, in a paradigm like RSVP Keyboard (Orhan et
154 al., 2012), or simply to fuse with the evidence gathered from the user (e.g., EEG, gaze, etc.).

155 The underlying models that drive the predictions are modular, and custom LM classes can
156 be created or modified to suit users' needs. BciPy 2.0 introduces inference by large language
157 models (LLMs) via the TextSlinger API, which allows the use of causal transformer models from
158 Hugging Face using the search algorithm from Gaines & Vertanen (2025). Wrapper classes such
159 as CausalLanguageModelAdapter handle the initialization of TextSlinger models for seamless
160 integration with BciPy. In addition to the causal LLM, BciPy 2.0 supports TextSlinger's
161 NGramLanguageModel class, which leverages the KenLM package (Heafield, 2011).

162 BciPy 2.0 also provides a UniformLanguageModel class, which returns an equal probability
163 distribution among all characters in the symbol set. This is useful when researchers want to
164 remove language model influence as a control condition or independent variable, and it serves
165 as an example of the methods required to create custom LanguageModel subclasses.

166 BciPy Simulator

167 A major advancement in BciPy 2.0 is the introduction of the simulator module. This module
168 allows researchers to use previously recorded typing data to systematically evaluate how
169 changes in parameters, signal models, and language models impact typing performance. The
170 simulator supports tasks such as customizing experiment parameters, conducting large-scale

171 comparisons of language model implementations, and testing multimodal evidence fusion
172 strategies. Additionally, the simulator framework can be extended to train new EEG signal
173 models.

174 As illustrated in Figure 4, the simulator architecture consists of several key components:

- 175 ■ The SimulatorTask (e.g., CopyPhrase),
- 176 ■ A TaskRunner for managing multiple iterations,
- 177 ■ A DataEngine for loading and querying data samples,
- 178 ■ A DataProcessor for formatting data for classification, and
- 179 ■ A Sampler that selects samples from the DataEngine using user-defined strategies.

180 The simulator collects metrics for each run and summarizes across all runs to assess performance.
181 The simulator provides both a graphical user interface for designing simulation parameters and
182 input sources, as well as a command line interface for scripting and automation.

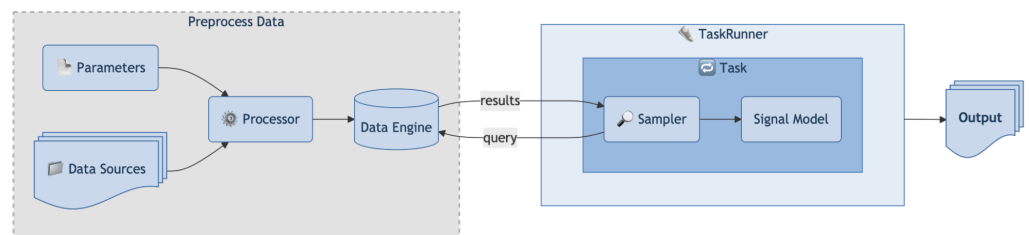


Figure 4: BciPy Simulator Architecture. The BciPy Simulator consists of several components that work together to simulate a typing task using previously recorded data. The SimulatorTask defines the task to be performed, such as CopyPhrase. The TaskRunner manages the execution of multiple iterations of the task, collecting metrics for each run. The DataEngine loads and queries data samples from the provided dataset, while the DataProcessor prepares the data to match the input format required by the classification model. The Sampler draws samples from the DataEngine based on a user-selected sampling strategy, such as random sampling or sequential sampling. The SignalModel and LanguageModel are used to classify the sampled data and provide predictions, respectively. Finally, the collected metrics are summarized across all runs to evaluate performance.

183 AI usage disclosure

184 The manuscript was written by the authors with the assistance of AI tools, including ChatGPT,
185 Claude, Grammarly, and GitHub Copilot. The AI tools were used to help edit text and to
186 assist with code review, documentation, testing, and formatting. The majority of BciPy core
187 functionality was developed by the authors exclusively. The authors reviewed and edited all
188 content generated by the AI tools to ensure accuracy and coherence.

Acknowledgements

We'd like to thank those who helped throughout the refactor, including Aida Fakhry, Ian Jackson, Julia Gangemi, Tales Imbiriba, Emma Sombers, Georgios Stratis, David Smith, Shijia Liu, Barry Oken, Deniz Erdogmus, Betts Peters, and Melanie Fried-Oken. In addition, we thank Steven Bedrick for his architectural and general advice on this significant update. This work was supported by NIH R01DC009834 and NSF IIS-1750193. Authors report no conflicts of interest.

References

- Booth, L., Asghar, A., & Bateson, A. (2023). PyBCI: A python package for brain-computer interface (BCI) design. *Journal of Open Source Software*, 8(92), 5706. <https://doi.org/10.21105/joss.05706>
- Gaines, D., & Vertanen, K. (2025). Adapting large language models for character-based augmentative and alternative communication. *Findings of the Association for Computational Linguistics: EMNLP 2025*, 15273–15291. <https://doi.org/10.18653/v1/2025.findings-emnlp.826>
- Gramfort, A., Luessi, M., Larson, E., Engemann, D. A., Strohmeier, D., Brodbeck, C., Parkkonen, L., & Hämäläinen, M. S. (2014). MNE software for processing MEG and EEG data. *Neuroimage*, 86, 446–460. <https://doi.org/10.1016/j.neuroimage.2013.10.027>
- Heafield, K. (2011). KenLM: Faster and smaller language model queries. *Proceedings of the Sixth Workshop on Statistical Machine Translation*, 187–197.
- Kothe, C., Shirazi, S. Y., Stenner, T., Medine, D., Boulay, C., Grivich, M. I., Artoni, F., Mullen, T., Delorme, A., & Makeig, S. (2025). The lab streaming layer for synchronized multimodal recording. *Imaging Neuroscience*. <https://doi.org/10.1162/IMAG.a.136>
- McKinney, W., & others. (2011). Pandas: A foundational python library for data analysis and statistics. *Python for High Performance and Scientific Computing*, 14(9), 1–9.
- Mei, J., Luo, R., Xu, L., Zhao, W., Wen, S., Wang, K., Xiao, X., Meng, J., Huang, Y., Tang, J., Cheng, L., Xu, M., & Ming, D. (2024). MetaBCI: An open-source platform for brain-computer interfaces. *Computers in Biology and Medicine*, 168. <https://doi.org/10.1016/j.combiomed.2023.107806>
- Memmott, T., Koçanaoğulları, A., Lawhead, M., Klee, D., Dudy, S., Fried-Oken, M., & Oken, B. (2021). BciPy: Brain-computer interface software in python. *Brain-Computer Interfaces*, 8(4), 137–153. <https://doi.org/10.1080/2326263x.2021.1878727>
- Orhan, U., Hild, K. E., Erdogmus, D., Roark, B., Oken, B., & Fried-Oken, M. (2012). RSVP keyboard: An EEG based typing interface. *2012 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 645–648. <https://doi.org/10.1109/ICASSP.2012.6287966>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Peirce, J. W. (2007). PsychoPy—psychophysics software in python. *Journal of Neuroscience Methods*, 162(1-2), 8–13. <https://doi.org/10.1016/j.jneumeth.2006.11.017>
- Renard, Y., Lotte, F., Gibert, G., Congedo, M., Maby, E., Delannoy, V., Bertrand, O., & Lécuyer, A. (2010). OpenViBE: An open-source software platform to design, test, and use brain-computer interfaces in real and virtual environments. *Presence*, 19(1), 35–53.

- 234 <https://doi.org/10.1162/pres.19.1.35>
- 235 Schalk, G., McFarland, D. J., Hinterberger, T., Birbaumer, N., & Wolpaw, J. R. (2004).
236 BCI2000: A general-purpose brain-computer interface (BCI) system. *IEEE Transactions on*
237 *Biomedical Engineering*, 51(6), 1034–1043. <https://doi.org/10.1109/TBME.2004.827072>
- 238 Singh, G., Chharia, A., Upadhyay, R., Kumar, V., & Longo, L. (2025). PyNoetic: A modular
239 python framework for no-code development of EEG brain-computer interfaces. *PLOS ONE*,
240 20(8), e0327791. <https://doi.org/10.1371/journal.pone.0327791>
- 241 Van Der Walt, S., Colbert, S. C., & Varoquaux, G. (2011). The NumPy array: A structure
242 for efficient numerical computation. *Computing in Science & Engineering*, 13(2), 22–30.
243 <https://doi.org/10.1109/mcse.2011.37>
- 244 Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D.,
245 Burovski, E., Peterson, P., Weckesser, W., Bright, J., & others. (2020). SciPy 1.0:
246 Fundamental algorithms for scientific computing in python. *Nature Methods*, 17(3),
247 261–272. <https://doi.org/10.1038/s41592-019-0686-2>
- 248 Wilkinson, M. D., Dumontier, M., Aalbersberg, I. J., Appleton, G., Axton, M., Baak, A.,
249 Blomberg, N., Boiten, E., Silva Santos, L. B. da, Bourne, P. E., & others. (2016). The
250 FAIR guiding principles for scientific data management and stewardship. *Scientific Data*,
251 3(1), 160018. <https://doi.org/10.1038/sdata.2016.18>
- 252 Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T.,
253 Louf, R., Funtowicz, M., & others. (2020). Transformers: State-of-the-art natural language
254 processing. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language*
255 *Processing: System Demonstrations*, 38–45. [https://doi.org/10.18653/v1/2020.emnlp-](https://doi.org/10.18653/v1/2020.emnlp-demos.6)
256 [demos.6](https://doi.org/10.18653/v1/2020.emnlp-demos.6)
- 257 Zhu, H., Beierholm, U., & Shams, L. (2024). BCI toolbox: An open-source python package
258 for the bayesian causal inference model. *PLOS Computational Biology*, 20(7), e1011791.
259 <https://doi.org/10.1371/journal.pcbi.1011791>